

✓ Algoritmos - Actividad Guiada 1

Nombre: David Castro Reyes

URL: <https://colab.research.google.com/drive/1JqfBFQFbfb39vkg9KIMqWj527tXl9uY?usp=sharing>

GITHUB: <https://github.com/davacast-ia/AlgoritmosOptimizacion>

✓ Torres de Hanoi con Divide y vencerás

```
def Torres_Hanoi(N, desde, hasta):
    if N == 1 :
        print("Lleva la ficha desde la torre" ,desde , " hasta ", hasta )

    else:
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )
        Torres_Hanoi(N-1, desde, 6-desde-hasta )
        print("Lleva la ficha desde la torre" ,desde , " hasta ", hasta )
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )

Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha desde la torre 1 hasta 3
Lleva la ficha desde la torre 1 hasta 2
Lleva la ficha desde la torre 3 hasta 2
Lleva la ficha desde la torre 1 hasta 3
Lleva la ficha desde la torre 2 hasta 1
Lleva la ficha desde la torre 2 hasta 3
Lleva la ficha desde la torre 1 hasta 3
```

```
#Sucesión de Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

8

✓ Devolución de cambio por técnica voraz

```
def cambio_monedas(N, SM):
    SOLUCION = [0]*len(SM) #SOLUCION = [0,0,0,0,..]
    ValorAcumulado = 0

    for i,valor in enumerate(SM):
        monedas = (N-ValorAcumulado)//valor
        SOLUCION[i] = monedas
        ValorAcumulado = ValorAcumulado + monedas*valor

    if ValorAcumulado == N:
        return SOLUCION

cambio_monedas(15,[25,10,5,1])
```

[0, 1, 1, 0]

✓ N-Reinas por técnica de vuelta atrás

```
def escribe(S):
    n = len(S)
    for x in range(n):
        print("")
        for i in range(n):
            if S[i] == x+1:
                print(" X " , end="")
            else:
                print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la misma fila
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i])) + " veces")
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

    #Verifica las diagonales
    for j in range(i+1, etapa +1 ):
        #print("Comprobando diagonal de " + str(i) + " y " + str(j))
        if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i
        escribe(solucion)
        print()

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

reinas(8)
```

```

X - - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- X - - - - -
- - - - -
X - X - - - -

- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- X - - - - -
X - - - - -

- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
- - - - -
X X - - - - -

```

✓ Viaje por el río. Programación dinámica

```

TARIFAS = [
[0,5,4,3,999,999,999],
[999,0,999,2,3,999,11],
[999,999, 0,1,999,4,10],
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in [9999]*N]
    RUTA = [ [""]*N for i in [""]*N]

    for i in range(0,N-1):
        RUTA[i][i] = i          #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0      #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k          #Anota que para ir de i a j hay que pasar por k
            PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

```

```

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
    if desde == hasta:
        #print("Ir a :" + str(desde))
        return ""
    else:
        return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
            ',' + \
            str(RUTA[desde][hasta] \
            )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

```

PRECIOS
[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

```

RUTA
[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']

```

```

La ruta es:
',0,2,5'

```

Reto propuesto: Encontrar los dos puntos más cercanos

Dado un conjunto de puntos, se trata de encontrar los dos puntos más cercanos.

Guía para el aprendizaje:

Suponer en 1D, o sea, una lista de números: [3403, 4537, 9089, 9746, 7259, ...]

Primer intento: Fuerza bruta

Calcular la complejidad. ¿Se puede mejorar?

Segundo intento: Aplicar Divide y Vencerás

Calcular la complejidad. ¿Se puede mejorar?

Extender el algoritmo a 2D: [(1122, 6175), (135, 4076), (7296, 2741), ...]

Extender el algoritmo a 3D.

```

import math
import random
from itertools import combinations

# ----- 1D -----
def distancia_1d(a, b):
    return abs(a - b)

def fuerza_bruta_1d(puntos):
    # O(n^2)
    n = len(puntos)
    mejor = (float('inf'), None, None)
    for i in range(n):
        for j in range(i + 1, n):
            d = distancia_1d(puntos[i], puntos[j])
            if d < mejor[0]:
                mejor = (d, puntos[i], puntos[j])
    return mejor

```

```

def divide_venceras_1d(puntos):
    # O(n log n)
    puntos_ordenados = sorted(puntos)

    def resolver(sub):
        n = len(sub)
        if n <= 3:
            return fuerza_bruta_1d(sub)
        medio = n // 2
        izq, der = sub[:medio], sub[medio:]
        mejor_izq = resolver(izq)
        mejor_der = resolver(der)
        mejor = mejor_izq if mejor_izq[0] <= mejor_der[0] else mejor_der
        d_frontera = distancia_1d(izq[-1], der[0])
        if d_frontera < mejor[0]:
            mejor = (d_frontera, izq[-1], der[0])
        return mejor

    return resolver(puntos_ordenados)

# ----- 2D -----
def distancia_2d(p, q):
    return math.hypot(p[0] - q[0], p[1] - q[1])

def fuerza_bruta_2d(puntos):
    # O(n^2)
    mejor = (float('inf'), None, None)
    for p, q in combinations(puntos, 2):
        d = distancia_2d(p, q)
        if d < mejor[0]:
            mejor = (d, p, q)
    return mejor

def divide_venceras_2d(puntos):
    # O(n log n)
    Px = sorted(puntos, key=lambda p: p[0])
    Py = sorted(puntos, key=lambda p: p[1])
    return _resolver_2d(Px, Py)

def _resolver_2d(Px, Py):
    n = len(Px)
    if n <= 3:
        return fuerza_bruta_2d(Px)
    medio = n // 2
    Px_izq, Px_der = Px[:medio], Px[medio:]
    conjunto_izq = set(Px_izq)
    Py_izq = [p for p in Py if p in conjunto_izq]
    Py_der = [p for p in Py if p not in conjunto_izq]
    mejor_izq = _resolver_2d(Px_izq, Py_izq)
    mejor_der = _resolver_2d(Px_der, Py_der)
    mejor = mejor_izq if mejor_izq[0] <= mejor_der[0] else mejor_der
    d = mejor[0]
    x_medio = Px[medio][0]
    franja = [p for p in Py if abs(p[0] - x_medio) < d]
    for i in range(len(franja)):
        for j in range(i + 1, min(i + 7, len(franja))):
            dist = distancia_2d(franja[i], franja[j])
            if dist < d:
                d = dist
                mejor = (dist, franja[i], franja[j])
    return mejor

# ----- Generación de datos (según indicación del profesor) -----
LISTA_1D = [random.randrange(1, 10000) for x in range(1000)]
LISTA_2D = [(random.randrange(1, 10000), random.randrange(1, 10000)) for x in range(1000)]

# ----- Resultados -----
print("Puntos más cercanos en 1D (fuerza bruta):      ", fuerza_bruta_1d(LISTA_1D))
print("Puntos más cercanos en 1D (divide y vencerás):", divide_venceras_1d(LISTA_1D))

print("Puntos más cercanos en 2D (fuerza bruta):      ", fuerza_bruta_2d(LISTA_2D))
print("Puntos más cercanos en 2D (divide y vencerás):", divide_venceras_2d(LISTA_2D))

Puntos más cercanos en 1D (fuerza bruta):      (0, 4548, 4548)
Puntos más cercanos en 1D (divide y vencerás): (0, 59, 59)

```

```
Puntos más cercanos en 2D (fuerza bruta): (8.54400374531753, (9972, 2601), (9975, 2609))  
Puntos más cercanos en 2D (divide y vencerás): (8.54400374531753, (9972, 2601), (9975, 2609))
```